Notes on: Programming with Arduino Uno_from_0

1.) 3.1 ARDUINO UNO board Block diagram

3.1 ARDUINO UNO board Block diagram

Introduction to the Block Diagram
- A block diagram simplifies a complex system.
- It shows the main functional units or blocks of the Arduino Uno.
- It illustrates how these blocks are connected and interact with each other.
- Understanding the block diagram is key to using the Arduino board effectively.
- It helps you understand what happens when you power the board, upload code, and run a program.

The main functional blocks of an Arduino Uno board are:
1. Microcontroller (The Brain)
2. Power Supply Section
3. Input/Output (I/O) Section
4. Programming and Communication Interface
5. Timing and Control Section

- ---

1. The Microcontroller (ATmega328P) - The Brain of the Board
- The Arduino Uno board is based on the ATmega328P microcontroller.
- This is a single chip that acts as the computer for the board.
- It executes the code you write and upload.
- It contains several key components inside it:

- Central Processing Unit (CPU)
- The core of the microcontroller; it performs calculations and makes decisions.
- It fetches instructions from the memory and executes them one by one.
- It controls all other parts of the microcontroller.

- Memory
- The microcontroller has three types of memory:
- 1. Flash Memory (32 KB)
- This is non-volatile memory; meaning it stores data even when power is off.
- Your program code (the **sketch**) is stored here.
- A small part of it is used by the bootloader; which helps in uploading new code without external hardware.
- Think of it as the hard disk of the Arduino.
- 2. SRAM (Static Random Access Memory) (2 KB)
- This is volatile memory; data is lost when power is turned off.
- It is used to store variables created and used by your program while it is running.
- It is much faster than Flash or EEPROM.
- Think of it as the RAM of the Arduino.
- 3. EEPROM (Electrically Erasable Programmable Read-Only Memory) (1 KB)
- This is non-volatile memory.
- Used for storing long-term data that should not be lost when the board is powered off.
- Examples: storing a sensor's calibration value or a device state.
- Think of it as a small, permanent storage file.

- ---

2. Power Supply Section - Providing Life to the Board
- This section manages the power for the entire board.
- The Arduino Uno can be powered from two main sources:
- 1. USB Port
- Provides 5V DC power directly from your computer.
- It is the most common way to power the board during development.
- 2. DC Power Jack (Barrel Jack)
- Accepts an external power supply; like a battery or AC-to-DC adapter.
- Recommended input voltage is 7V to 12V DC.

- Voltage Regulators
- The board has voltage regulators to provide stable and constant voltage.
- The input from the DC jack can be high (e.g., 9V or 12V).
- The microcontroller and other components need a stable 5V to work.
- The 5V regulator converts the input voltage (from DC jack) to a clean 5V.
- The board also has a 3.3V regulator; which provides 3.3V for sensors and components that need a lower voltage.
- This section includes power pins on the board:
- Vin: Input voltage pin for external power.
- 5V: Provides regulated 5V output.
- 3.3V: Provides regulated 3.3V output.
- GND: Ground pins; the common reference point for the circuit.

- ---

3. Input/Output (I/O) Section - Interacting with the World
- This section consists of pins that connect the microcontroller to external components.
- These are the physical connection points for sensors, motors, LEDs, and other devices.

- Digital I/O Pins
- The Uno has 14 digital I/O pins; labeled 0 to 13.
- They can operate in two modes:
- Input mode: To read a signal from a sensor (e.g., a push button). They can detect two states: HIGH (5V) or LOW (0V).
- Output mode: To send a signal to an actuator (e.g., turn an LED on or off). They can output two states: HIGH (5V) or LOW (0V).
- These pins are controlled using programming functions like pinMode(), digitalWrite(), and digitalRead().
- PWM Pins: Some digital pins (marked with '~', e.g., 3, 5, 6, 9, 10, 11) support Pulse Width Modulation (PWM).
- PWM allows you to simulate an analog output; for example, to control the brightness of an LED or the speed of a motor.

- Analog Input Pins
- The Uno has 6 analog input pins; labeled A0 to A5.
- These pins are used to read analog signals; which are voltages that can vary continuously.
- For example, reading the value from a potentiometer or a temperature sensor.
- They have a 10-bit Analog-to-Digital Converter (ADC).
- This means they can measure the input voltage (from 0V to 5V) and map it to a digital value from 0 to 1023.
- These pins can also be used as digital I/O pins if needed.

- Specialized Communication Pins
- These pins are used for communication between the Arduino and other devices or microcontrollers.
- 1. Serial (UART): Pins 0 (RX - Receive) and 1 (TX - Transmit).
- Used for communication with a computer or other serial devices.
- These pins are connected to the USB-to-Serial converter chip.
- 2. I2C (Inter-Integrated Circuit): Pins A4 (SDA) and A5 (SCL).
- A two-wire protocol to communicate with multiple sensors and devices on the same bus.

- 3. SPI (Serial Peripheral Interface): Pins 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK).
- A high-speed communication protocol often used for SD cards and display screens.

- ---

4. Programming and Communication Interface
- This section acts as a bridge between your computer and the ATmega328P microcontroller.

- USB Port (Type B)
- It serves two main purposes:
- 1. To upload your compiled code (sketch) from the Arduino IDE on your computer to the microcontroller's flash memory.
- 2. To provide power to the board.
- 3. To establish serial communication for sending/receiving data between the Arduino and your computer (using the Serial Monitor).

- USB-to-Serial Converter Chip
- On most Uno boards, this is a separate microcontroller (ATmega16U2).
- Its only job is to convert the USB signals from your computer into serial signals (TX/RX) that the main ATmega328P microcontroller can understand, and vice-versa.
- This chip allows your computer to see the Arduino as a virtual serial port (COM port).

- TX/RX LEDs
- Two small LEDs on the board, labeled TX (Transmit) and RX (Receive).
- They blink whenever data is being sent or received over the serial communication line.
- This is a very useful visual indicator for debugging.

- ---

5. Timing and Control Section
- This section provides the essential clock signal and control features for the microcontroller.

- 16 MHz Crystal Oscillator
- This is a tiny electronic component that generates a very precise and stable clock signal.
- It acts like the **heartbeat** of the microcontroller.
- It provides a 16 million cycles per second (16 MHz) clock pulse.
- The CPU executes instructions in sync with this clock. A faster clock means faster processing.

- Reset Button
- A physical push button on the board.
- When pressed, it temporarily connects the reset pin to ground.
- This restarts the microcontroller, and the program stored in the flash memory starts running again from the beginning.
- It is useful for restarting your project without unplugging the power.
- There is also a reset pin that allows an external circuit (like a shield) to reset the board.

- ---

6. Other On-board Components

- Power LED
- A green LED that lights up whenever the Arduino board is connected to a power source.
- It gives a quick visual confirmation that the board is powered on.

- Pin 13 LED (L)
- A built-in LED connected to digital pin 13.
- It is very convenient for beginners to test if the board is working.
- The classic **Blink** example sketch, the **Hello World** of Arduino, uses this LED.
- You can test your first program without connecting any external components.

- ICSP Header (In-Circuit Serial Programming)
- A set of 6 pins, usually in a 2x3 configuration.
- It is an advanced feature.
- It allows you to program the microcontroller's memory directly, bypassing the bootloader.
- This is used for burning a new bootloader or for programming the chip using an external programmer.

- ---

Summary - How the Blocks Work Together
1. You connect the Arduino to your computer via USB.
2. The Power Supply section gets 5V from the USB, the Power LED turns on, and the voltage regulators ensure stable 5V/3.3V for all components.
3. The 16 MHz Crystal Oscillator starts providing the clock signal to the ATmega328P microcontroller.
4. You write a program (sketch) in the Arduino IDE and click **Upload**.
5. The program is sent through the USB cable.
6. The USB-to-Serial converter chip (ATmega16U2) translates this data into serial format.
7. The serial data is sent to the main ATmega328P microcontroller.
8. The bootloader on the ATmega328P receives this code and writes it to the Flash Memory.
9. After uploading, the microcontroller resets and starts executing the new code from Flash Memory.
10. The code reads data from sensors connected to Input pins and controls actuators connected to Output pins.
11. Your project comes to life.

2.) 3.2 Sketch Structure

3.2 Sketch Structure

1. Introduction to an Arduino Sketch

- An Arduino program is called a **Sketch**.
- It is the set of instructions you write; to tell the Arduino Uno board what to do.
- Think of it like a recipe for a dish; the Arduino is the cook.
- Every sketch has a specific structure; a rule you must follow.
- This structure makes the code predictable; and easy for the microcontroller to run.
- You write sketches in the Arduino IDE (Integrated Development Environment).
- The language is based on C/C++.
- After writing, you upload the sketch to the Arduino board's memory.
- The board then runs your instructions.

2. The Two Essential Functions

- Every Arduino sketch MUST have two special functions.
- These are setup() and loop().
- A function is a block of code; that performs a specific task.
- Without these two functions; your sketch will not compile or run.
- They are the foundation of any Arduino program.

2.1 The setup() Function

- The setup() function is the first part of your code to run.
- It runs only ONCE.
- It runs after the Arduino board is powered on; or after the reset button is pressed.
- Its main purpose is initialization; preparing the board for its main task.

- What goes inside setup()?
- - Pin Configuration:
- - Telling the Arduino if a pin is an INPUT or an OUTPUT.
- - Uses the pinMode() function for this.
- - Example: pinMode(13, OUTPUT); - configures pin 13 as an output.
- - You learned about the pins in the Arduino block diagram (Topic 3.1).

- - Library Initialization:
- - Starting up any libraries you are using.
- - Libraries are pre-written code for complex tasks; like controlling an LCD screen.

- - Serial Communication Setup:
- - If you want the Arduino to talk to your computer.
- - You start the serial communication here.
- - Uses Serial.begin(9600);. The number 9600 is the communication speed (baud rate).

- - Variable Initialization:
- - Giving initial values to variables.

- Structure of setup():
- void setup() {
- // your setup code goes here, to run once:
- pinMode(13, OUTPUT);
- }
- void means the function does not return any value.
- The code is written between the curly braces {}.

2.2 The loop() Function

- The loop() function runs right after setup() is finished.
- As the name suggests, it runs over and over again.
- It loops continuously; forever.
- This is the main part of your program; where the primary action happens.
- The microcontroller on the Arduino board spends all its time running the code in loop().

- What goes inside loop()?
- - Reading Inputs:
- - Checking the state of a sensor; or a button.
- - Example: digitalRead(2); - reads the voltage on pin 2.

- - Processing Data:
- - Performing calculations; making decisions based on inputs.
- - Using operators and control statements (like if-else).

- - Controlling Outputs:
- - Turning an LED on or off; moving a motor.
- - Example: digitalWrite(13, HIGH); - sends 5V to pin 13 to turn an LED on.

- Structure of loop():
- void loop() {
- // your main code goes here, to run repeatedly:
- digitalWrite(13, HIGH); // turn the LED on
- delay(1000); // wait for a second
- digitalWrite(13, LOW); // turn the LED off
- delay(1000); // wait for a second
- }
- This example code will blink an LED connected to pin 13.
- It will repeat this blinking action forever.

• Analogy for setup() and loop():
• Imagine preparing to cook a meal.
• setup() is like washing vegetables, gathering ingredients, and preheating the oven. You do this only once at the beginning.
• loop() is the main cooking process, like stirring the pot continuously or checking if the food is cooked every few minutes. This repeats until you stop.

3. Other Optional (but important) Structural Components

• A sketch can have more than just setup() and loop().
• These parts help organize your code; and make it more powerful.

3.1 Variable and Constant Declaration Area

• This is usually at the very top of the sketch; before setup().
• Here you declare **global** variables.
• A global variable can be used anywhere in your sketch; inside setup(), loop(), or any other function.
• Declaring variables at the top makes the code easier to read.
• You will learn about data types (like int, char) in the next topic (3.3).
• Example:
• int ledPin = 13; // This declares a variable to hold the pin number.

3.2 Including Libraries

• Located at the very top of the sketch; even before variables.
• Uses the #include directive.
• Libraries provide extra functionality; for things like sensors, motors, displays.
• It's like adding a new chapter to your recipe book.
• Example:
• #include <Servo.h> // Includes the library to control servo motors.

3.3 User-Defined Functions

• You can create your own functions; besides setup() and loop().
• This helps to break down complex tasks into smaller, manageable parts.
• It makes the code cleaner; and easier to debug.
• You can call your custom function from inside loop() or setup().
• These are usually written below the loop() function.
• Example:
• void blinkLED() {
• digitalWrite(13, HIGH);
• delay(500);
• digitalWrite(13, LOW);
• delay(500);
• }

3.4 Comments

• Comments are notes in your code for humans to read.
• The compiler ignores them completely.
• They are used to explain what the code does.
• This is very important for understanding your own code later; or for others to understand it.
• There are two types of comments:
• - Single-line comment: Starts with //.
• - // This is a single-line comment.
• - Multi-line comment: Starts with /* and ends with */.
• - /*
• - This is a
• - multi-line comment.

- */

4. Complete Sketch Structure Example

- Here is a template showing all the parts together.

- // 1. Include Libraries (Optional)
- #include <SomeLibrary.h>

- // 2. Global Variable and Constant Declarations (Optional)
- const int LED_PIN = 13; // Use 'const' for values that don't change
- int sensorValue;

- // 3. The Setup Function (Mandatory)
- // Runs only once when the program starts.
- void setup() {
- // Initialize pin modes
- pinMode(LED_PIN, OUTPUT);

- // Start serial communication
- Serial.begin(9600);
- }

- // 4. The Loop Function (Mandatory)
- // Runs continuously after setup() is complete.
- void loop() {
- // Main logic of the program
- doSomething(); // Calling a user-defined function
- delay(100);
- }

- // 5. User-Defined Functions (Optional)
- // Your own custom functions to organize code.
- void doSomething() {
- // Code for a specific task goes here
- digitalWrite(LED_PIN, HIGH);
- }

- Summary for Revision:
- - Sketch: An Arduino program.
- - Structure: The way the code is organized.
- - Mandatory Functions: void setup() and void loop().
- - setup(): Runs once. For initialization. e.g., pinMode().
- - loop(): Runs repeatedly. For the main program logic. e.g., digitalWrite(), digitalRead().
- - Optional Parts: Library includes (#include), global variables, user-defined functions, comments (//, /* */).
- - Code Execution Flow: Global variables declared -> setup() runs once -> loop() runs forever.

3.) 3.3 Data types & Built in Constants

3.3 Data Types & Built-in Constants

- Introduction to Data Types

- A data type specifies the type of data a variable can store; like a number, character, or true/false value.

• It tells the compiler two things:
• 1. How much memory to allocate for the variable.
• 2. What kind of data can be stored in that memory.
• Think of it like a container; a small box for a small item, a large box for a large item.
• Using the correct data type is crucial on microcontrollers like Arduino Uno; because memory is very limited (2KB of SRAM).
• It prevents errors and makes your code efficient.

• Variable Declaration

• To use a variable, you must first declare it.
• Declaration tells the compiler the variable's type and its name.
• Syntax: dataType variableName;
• Example: int counter;
• You can also initialize it (give it a starting value) during declaration.
• Syntax: dataType variableName = value;
• Example: int counter = 0;

• --- Main Data Types in Arduino ---

• 1. Integer Data Types (for whole numbers)

• A. byte
• Stores a number from 0 to 255.
• Unsigned; meaning it only holds positive values.
• Size: 1 byte (8 bits).
• Range: 0 to 255.
• Use case: Storing data from a sensor that gives values in this range; or for controlling PWM brightness.
• Example: byte ledBrightness = 200;

• B. int (Integer)
• The most common data type for numbers.
• Stores whole numbers; can be positive or negative.
• Size: 2 bytes (16 bits) on Arduino Uno.
• Range: -32,768 to 32,767.
• Use case: General purpose counting; storing sensor readings.
• Example: int sensorValue;
• Note: If a value goes beyond its range (overflow); it will **wrap around** to the other end.
• e.g., 32767 + 1 becomes -32768. This is a common bug.

• C. unsigned int
• Same as int but stores only positive values.
• Size: 2 bytes (16 bits).
• Range: 0 to 65,535.
• Use case: When you know the value will never be negative; doubling the positive range.
• Example: unsigned int timeElapsed;

• D. long
• Used for very large integer values.
• Size: 4 bytes (32 bits).
• Range: -2,147,483,648 to 2,147,483,647.
• Use case: Storing the return value from the millis() function; which counts milliseconds since the Arduino started.
• Example: long currentTime = millis();

• E. unsigned long
• Stores very large positive integer values.
• Size: 4 bytes (32 bits).

• Range: 0 to 4,294,967,295.
• Use case: Also used with millis() for timing calculations that span long durations.
• Example: unsigned long previousTime;

• 2. Floating-Point Data Types (for numbers with decimals)

• A. float
• Stores numbers with a decimal point (fractional numbers).
• Size: 4 bytes (32 bits).
• Precision: 6-7 decimal digits.
• Use case: Storing sensor readings that require precision; like temperature or voltage.
• Example: float temperature = 28.5;
• Note: Floating-point math is slower than integer math on Arduino.

• B. double
• Double-precision floating-point number.
• On more powerful computers, it has more precision than float.
• IMPORTANT: On Arduino Uno (AVR-based boards); double is the same size as float (4 bytes).
• There is no extra precision gain; so float is preferred.
• Example: double piValue = 3.14159; (acts just like a float on Uno).

• 3. Boolean Data Type

• boolean
• Can only hold one of two values: true or false.
• Internally, false is 0 and true is 1 (or any non-zero number).
• Size: 1 byte.
• Use case: Storing states; like a switch being on or off; tracking conditions in loops.
• Example: boolean buttonPressed = false;

• 4. Character Data Type

• char
• Stores a single character.
• Characters are enclosed in single quotes ' '.
• Size: 1 byte.
• It actually stores the number corresponding to the character in the ASCII table.
• 'A' is stored as 65; 'a' is 97.
• Range: -128 to 127.
• Use case: Storing a single character received from serial communication.
• Example: char command = 's';

• unsigned char
• Same as byte; a single byte of information.
• Size: 1 byte.
• Range: 0 to 255.
• Example: unsigned char dataPacket = 150;

• 5. Special Data Types

• void
• A special type that means **nothing** or **no type**.
• Used in two main places:
• 1. A function that returns no value: void setup() { ... }
• 2. A function that takes no parameters: int readSensor(void) { ... }

• String (with a capital S)
• This is not a fundamental data type, but an object.
• It represents a sequence of characters (text).

• More flexible than character arrays but uses more memory and can cause issues.
• Use case: Handling text from serial monitor or an LCD display.
• Example: String message = **Hello Arduino**;

• array
• A collection of variables of the same data type.
• Example: int sensorReadings[5]; // an array to hold 5 integer readings.
• We will discuss this more in later topics.

• --- Built-in Constants ---

• Constants are pre-defined variables in the Arduino language.
• Their values do not change during program execution.
• They make code easier to read and understand.

• 1. Digital I/O Constants

• HIGH | LOW
• Represents the two states of a digital pin.
• HIGH: Represents 5V (or 3.3V on some boards); means the pin is ON.
• LOW: Represents 0V (GND); means the pin is OFF.
• Used with digitalWrite() to turn pins on/off.
• Used with digitalRead() to check a pin's state.
• Example: digitalWrite(13, HIGH);

• INPUT | OUTPUT | INPUT_PULLUP
• Used with the pinMode() function in setup() to configure a pin's behavior.
• INPUT: Configures the pin to read a signal; like from a sensor or a button.
• OUTPUT: Configures the pin to send a signal; like to an LED or a motor driver.
• INPUT_PULLUP: A special input mode; it uses an internal **pull-up** resistor.
• This is very useful for reading buttons without adding external resistors.
• Example: pinMode(7, INPUT_PULLUP);
• Example: pinMode(13, OUTPUT);

• 2. Boolean Constants

• true | false
• These are the only two values for a boolean variable.
• true is logically equivalent to 1.
• false is logically equivalent to 0.
• Used for logical checks and setting boolean variables.
• Example: if (buttonPressed == true) { ... }

• 3. Pin-related Constants

• LED_BUILTIN
• A constant that refers to the pin number of the on-board LED.
• On the Arduino Uno, this is pin 13.
• Using LED_BUILTIN makes your code more portable to other Arduino boards.
• Example: pinMode(LED_BUILTIN, OUTPUT);

• 4. Integer Constant Formats
• You can write integer numbers in different formats.
• Decimal (base 10): 123 (standard format)
• Binary (base 2): B1111011 (starts with 'B')
• Octal (base 8): 0173 (starts with '0')
• Hexadecimal (base 16): 0x7B (starts with '0x')

• Choosing the Right Data Type: A Summary

- Always choose the smallest data type that can hold your required range of values.
- For a counter that never exceeds 200, use byte instead of int. This saves memory.
- For numbers that will never be negative, use unsigned types to get a larger positive range.
- Avoid using float unless you absolutely need decimal values, as it's slower.
- Use long or unsigned long for time-related functions like millis().
- This practice of efficient memory usage is essential for embedded systems like Arduino.

4.) 3.4 Operators: Arithmetic, Bitwise, Compound, Comparison, and Boolean

3.4 Operators in Arduino Programming

Introduction to Operators
- Operators are special symbols; they perform operations on variables and values.
- The values operators work on are called operands.
- Example: In x + y, x and y are operands; + is the operator.
- Mastering operators is essential; for writing any logic in your Arduino sketch.
- We will cover five main types of operators.

1. Arithmetic Operators
- Used for performing mathematical calculations.
- They work just like in basic mathematics.
- Very common in almost every Arduino sketch; for processing sensor data or timing.

- Addition (+)
- Adds two operands.
- Example: int sum = 10 + 5; // sum will be 15
- int sensorValue = analogRead(A0);
- int adjustedValue = sensorValue + 50;

- Subtraction (-)
- Subtracts the second operand from the first.
- Example: int difference = 10 - 5; // difference will be 5
- int calibratedReading = rawReading - offset;

- Multiplication (*)
- Multiplies two operands.
- Example: int product = 10 * 5; // product will be 50
- float voltage = sensorValue * (5.0 / 1023.0);

- Division (/)
- Divides the first operand by the second.
- Important: Behavior depends on data types.
- Integer Division: If both operands are integers, the result is an integer; decimal part is discarded.
- Example: int result = 7 / 2; // result is 3, not 3.5
- Floating-Point Division: If at least one operand is a float, the result is a float.
- Example: float result = 7.0 / 2; // result is 3.5

- Modulo (%)
- Returns the remainder of a division.
- Very useful for tasks that need to cycle or repeat.
- Example: int remainder = 7 % 2; // remainder is 1
- Useful for checking if a number is even or odd: if (number % 2 == 0)
- Useful for cycling through a fixed number of states: state = (state + 1) % 4; // state will cycle 0, 1, 2, 3, 0...

2. Bitwise Operators
- Operate on variables at the binary level; on individual bits.
- Extremely useful in embedded systems and IoT; for controlling hardware registers.
- Allows for fast and memory-efficient manipulation of data.
- Let's assume int a = 5; (0101 in binary) and int b = 3; (0011 in binary).

- Bitwise AND (&)
- Result bit is 1; only if both corresponding bits are 1.
- Often used for **masking**; to check or clear a specific bit.
- Example: int result = a & b; // 0101 & 0011 = 0001 (decimal 1)
- Use case: Check if the 3rd bit of a variable x is set: if (x & B00001000)

- Bitwise OR (|)
- Result bit is 1; if at least one of the corresponding bits is 1.
- Often used to **set** a specific bit to 1; without changing others.
- Example: int result = a | b; // 0101 | 0011 = 0111 (decimal 7)
- Use case: Turn on the 5th pin representation in a control register: PORTD = PORTD | B00100000;

- Bitwise XOR (^)
- Result bit is 1; if the corresponding bits are different.
- Often used to **toggle** or **flip** a specific bit.
- Example: int result = a ^ b; // 0101 ^ 0011 = 0110 (decimal 6)
- Use case: Toggle an LED connected to pin 5: PORTD = PORTD ^ B00100000;

- Bitwise NOT (~)
- Inverts all the bits of the operand.
- Changes every 0 to 1; and every 1 to 0.
- Example: int result = ~a; // ~00000101 becomes 11111010
- Note: The result depends on the size of the data type (e.g., 16 bits for int).

- Left Shift (<<)
- Shifts the bits of the left operand; to the left by the number of positions specified by the right operand.
- Equivalent to multiplying by 2 for each shift position.
- Example: int result = a << 2; // 00000101 << 2 becomes 00010100 (decimal 20)
- 5 * (2^2) = 5 * 4 = 20.

- Right Shift (>>)
- Shifts the bits of the left operand; to the right.
- Equivalent to dividing by 2 for each shift position.
- Example: int result = a >> 1; // 00000101 >> 1 becomes 00000010 (decimal 2)
- 5 / 2 = 2 (integer division).

3. Compound Assignment Operators
- A shorthand way; to combine a standard operator with an assignment operator (=).
- Makes code shorter; more readable; and sometimes more efficient.
- General form: variable op= value; is same as variable = variable op value;

- += (Addition assignment)
- x += 5; is the same as x = x + 5;

- -= (Subtraction assignment)
- x -= 5; is the same as x = x - 5;

- *= (Multiplication assignment)
- x *= 5; is the same as x = x * 5;

- /= (Division assignment)
- x /= 5; is the same as x = x / 5;

- %= (Modulo assignment)
- x %= 5; is the same as x = x % 5;

- Bitwise compound operators also exist: &=, |=, ^=, <<=, >>=.
- Example: PORTD |= B00100000; sets the 5th bit of PORTD.

4. Comparison (Relational) Operators
   - Used to compare two operands.
   - The result of a comparison is always a boolean value; true (1) or false (0).
   - These are the foundation for decision-making in code; used in if, while, for statements (Topic 3.5).
   - Let's assume int x = 5; and int y = 10;.

   - Equal to (==)
   - Checks if two operands are equal.
   - CRITICAL: Use == for comparison, not = (which is for assignment). A common beginner mistake.
   - Example: (x == 5) evaluates to true.
   - Example: (x == y) evaluates to false.

   - Not equal to (!=)
   - Checks if two operands are not equal.
   - Example: (x != y) evaluates to true.

   - Greater than (>)
   - Checks if the left operand is greater than the right.
   - Example: (y > x) evaluates to true.

   - Less than (<)
   - Checks if the left operand is less than the right.
   - Example: (x < y) evaluates to true.

   - Greater than or equal to (>=)
   - Checks if the left operand is greater than or equal to the right.
   - Example: (x >= 5) evaluates to true.

   - Less than or equal to (<=)
   - Checks if the left operand is less than or equal to the right.
   - Example: (y <= 10) evaluates to true.

5. Boolean (Logical) Operators
   - Used to combine multiple comparison expressions.
   - The result is also a boolean value: true or false.
   - Essential for creating complex conditions in control statements.

   - Logical AND (&&)
   - Returns true; only if both expressions are true.
   - Example: (x < 10 && y > 5) // (true && true) results in true.
   - Use case: if (temperature > 25 && humidity < 60) // Take action only if it's hot AND dry.

   - Logical OR (||)
   - Returns true; if at least one of the expressions is true.
   - Example: (x > 10 || y > 5) // (false || true) results in true.
   - Use case: if (button1Pressed || button2Pressed) // Take action if either button is pressed.

   - Logical NOT (!)
   - Inverts the boolean value of an expression.
   - Makes a true expression false; and a false expression true.
   - Example: !(x == y) // x == y is false, so !(false) results in true.
   - Use case: while (!isTaskComplete) // Loop until the task is complete.

Summary of Operators
- Arithmetic operators for math.
- Bitwise operators for low-level hardware control.
- Compound operators for writing shorter code.
- Comparison and Boolean operators for making decisions.
- These operators are fundamental tools; you will use them in every Arduino project to control program flow and manipulate data from sensors and actuators.

5.) 3.5 Control statements and Loops

3.5 Control statements and Loops

Introduction to Control Flow

- In programming, code usually runs from top to bottom, one line at a time.
- Control statements change this default flow of execution.
- They allow your program to make decisions or repeat actions.
- This gives your Arduino sketch intelligence; it can react to sensor inputs.
- Two main types of control statements are:

1- Conditional Statements (for making decisions).
2- Loops (for repetition or iteration).

1- Conditional Statements (Decision Making)

- These statements execute a block of code only if a certain condition is met.
- They are the **brains** of your Arduino program.
- They use comparison (==, !=, <, >) and boolean (&&, ||, !) operators to evaluate conditions.

- if Statement
- The most basic decision-making statement.
- Purpose: Executes a block of code if its condition is true.
- If the condition is false, the code block is skipped.
- Syntax:

```
if (condition) {
// code to run if condition is true
}
```

- Example: Turn on an LED if a button is pressed.

```
int buttonState = digitalRead(2);
if (buttonState == HIGH) {
digitalWrite(13, HIGH); // Turn LED on
}
```

- Analogy: **If it is hot outside, turn on the fan**.

- if-else Statement
- An extension of the if statement.
- Purpose: Executes one block of code if the condition is true, and another block if it is false.
- It provides an alternative path for the program.
- Syntax:

```
if (condition) {
// code to run if condition is true
} else {
// code to run if condition is false
}
```

- Example: Turn an LED ON if a button is pressed, otherwise turn it OFF.

```
int buttonState = digitalRead(2);
```

```
if (buttonState == HIGH) {
digitalWrite(13, HIGH); // LED is ON
} else {
digitalWrite(13, LOW); // LED is OFF
}
```

- Analogy: **If you pass the exam, you get a new phone; else, you study more**.

- if-else if-else Ladder
- Used for checking multiple conditions.
- Purpose: Selects one block of code to execute from several options.
- The conditions are checked from top to bottom.
- The first condition that is true gets its code block executed.
- If no conditions are true, the final else block is executed (if present).
- Syntax:

```
if (condition1) {
// code for condition1
} else if (condition2) {
// code for condition2
} else {
// code if no conditions are true
}
```

- Example: Checking a temperature sensor reading.

```
int temp = analogRead(A0);
if (temp > 400) {
// Turn on a red LED (Too Hot)
} else if (temp > 200) {
// Turn on a yellow LED (Normal)
} else {
// Turn on a blue LED (Too Cold)
}
```

- switch-case Statement
- An efficient alternative to a long if-else if ladder.
- Purpose: Compares a variable against a list of specific, constant values (cases).
- It is often cleaner and faster for multi-way branching.
- Keywords used: switch, case, break, default.
- break: This keyword is very important. It stops the execution inside the switch block. Without it, the code **falls through** to the next case.
- default: This block runs if no other case matches the variable. It is optional.
- Syntax:

```
switch (variable) {
case value1:
// code for value1
break;
case value2:
// code for value2
break;
default:
// code if no case matches
break;
}
```

- Example: Controlling a motor based on a character received.

```
char command = Serial.read();
switch (command) {
case 'F':
// Move motor forward
break;
case 'B':
// Move motor backward
```

```
break;
case 'S':
// Stop motor
break;
default:
// Invalid command
break;
}
```

2- Loops (Iterative Statements)

- Loops are used to execute a block of code repeatedly.
- Essential for tasks like continuously reading a sensor or blinking an LED.
- The main loop in every Arduino sketch, `void loop() { ... }`, is itself an example of an infinite loop.

- `for` Loop
- Used when you know exactly how many times you want to repeat the code.
- It is a **count-controlled** loop.
- It has three parts in its parenthesis, separated by semicolons.

1- Initialization: Runs once at the start (e.g., `int i = 0`). Sets up a loop counter variable.
2- Condition: Checked before each repetition. The loop continues as long as this is true (e.g., `i < 10`).
3- Increment/Decrement: Runs at the end of each repetition. Changes the counter variable (e.g., `i++`).

- Syntax:

```
for (initialization; condition; increment) {
// code to be repeated
}
```

- Example: Blink an LED exactly 3 times.

```
for (int i = 0; i < 3; i++) {
digitalWrite(13, HIGH);
delay(250);
digitalWrite(13, LOW);
delay(250);
}
```

- `while` Loop
- Repeats a block of code as long as a specified condition is true.
- It is a **condition-controlled** loop.
- The condition is tested *before* executing the loop body.
- If the condition is false initially, the loop body will never execute.
- Warning: You must have code inside the loop that can eventually make the condition false, otherwise you create an infinite loop.
- Syntax:

```
while (condition) {
// code to be repeated
// update statement for the condition
}
```

- Example: Keep reading a sensor until its value is over 500.

```
int sensorValue = 0;
while (sensorValue <= 500) {
sensorValue = analogRead(A0);
delay(100); // Wait a bit before next reading
}
// Now sensorValue is greater than 500
```

- `do-while` Loop
- Similar to the `while` loop, but with one key difference.
- The condition is tested *after* executing the loop body.
- This guarantees that the loop body is executed at least one time.
- This loop is less common in Arduino programming but is useful in certain situations.

• Syntax:

```
do {
// code to be repeated
} while (condition);
```

• Example: Read a password from user until it is correct. The code must run at least once to get the first input.

```
int password;
do {
password = readPassword(); // A custom function to get password
} while (password != 1234);
```

3- Loop Control Statements

• These statements alter the normal flow of execution inside a loop.

• break Statement
• Purpose: To terminate a loop (for, while, do-while) or a switch statement immediately.
• When break is executed, the program control jumps to the statement immediately following the loop.
• Use case: Stop a search loop once the target item has been found.
• Example:

```
for (int i = 0; i < 100; i++) {
if (analogRead(A0) > 900) {
// High value detected, no need to loop further
break;
}
delay(10);
}
// Code continues here after the break
```

• continue Statement
• Purpose: To skip the rest of the current iteration of a loop and begin the next iteration.
• It does not terminate the entire loop, only the current pass.
• In for loops, it jumps to the increment/decrement part.
• In while and do-while loops, it jumps to the condition test.
• Use case: Ignore certain values during processing.
• Example: Sum only the even numbers from 0 to 9.

```
int sum = 0;
for (int i = 0; i < 10; i++) {
if (i % 2 != 0) { // If i is odd
continue; // Skip the rest of this iteration
}
sum = sum + i; // This line only runs for even i
}
```

• goto Statement
• Purpose: Provides an unconditional jump from the goto to a labeled statement in the same function.
• A label is a valid identifier followed by a colon (:).
• Syntax:

```
goto label;
...
label:
// statements
```

• Warning: The use of goto is highly discouraged in modern programming.
• It can make the code logic very confusing, hard to read, and difficult to debug (known as **spaghetti code**).
• Almost any problem solved with goto can be solved more elegantly with if, while, for, break, or by writing a function.
• It is included here for completeness of the C++ language features available in Arduino.

6.) 3.6 Functions and library functions (User defined functions, Library functions: I/O Functions: digitalRead, digitalWrite, pinMode, analogRead, analogWrite, analogReference. Char functions: isAlpha, isAlphaNumeric, isDigit, isHexadecimalDigit, isSpace, isWhitespace, isUpperCase, isLowerCase. Math Functions: abs, constrain, max, min, pow, sqrt)

3.6 Functions and library functions

• Introduction to Functions
• A function is a named block of code; it performs a specific task.
• Think of it as a mini-program inside your main program.
• Using functions makes your code organized; easier to read; and reusable.
• It helps avoid writing the same code multiple times.
• In Arduino programming, we use two main types of functions:
• 1. User-Defined Functions (Functions you create).
• 2. Library Functions (Functions provided by Arduino).

• User-Defined Functions (UDF)
• These are functions created by you, the programmer, for specific needs.
• They help break down complex problems into smaller, manageable parts.
• General Structure (Syntax):
• returnType functionName(parameter1, parameter2)
• {
• // function body: code that performs the task
• return value; // Used if returnType is not void
• }
• Explanation of each part:
• - returnType: The data type of the result the function sends back (e.g., int, float). Use void if it returns nothing.
• - functionName: A unique name you choose for the function.
• - parameters: Input values the function receives to work with. They are passed in parentheses ().
• - function body: The statements inside the curly braces {} that are executed when the function is called.
• - return: A keyword that sends a value back to the code that called the function.

• Example of a User-Defined Function:
• A function to add two integers.
•
• // Function Definition
• int addNumbers(int x, int y)
• {
• int sum = x + y;
• return sum; // Returns the calculated sum
• }
•
• // How to call this function in your code
• void loop()
• {
• int result = addNumbers(5, 3); // result will now hold the value 8
• }

• Library Functions
• Pre-written, built-in functions provided by the Arduino core libraries.

• They save you time and effort for common tasks like I/O or math.
• You don't need to know how they work internally; just how to use them.
• We will focus on three important sets of library functions.

• 1. I/O Functions (Input/Output)
• These functions control the Arduino's physical pins; allowing it to interact with electronic components.

• pinMode(pin, mode)
• - Purpose: Configures a pin to act as either an INPUT or an OUTPUT.
• - Must be called in setup() before using the pin.
• - pin: the number of the pin to configure (e.g., 13).
• - mode: INPUT, OUTPUT, or INPUT_PULLUP.
• - Example: pinMode(7, OUTPUT); // Sets digital pin 7 as an output.

• digitalWrite(pin, value)
• - Purpose: Writes a digital value (HIGH or LOW) to a pin configured as OUTPUT.
• - pin: the pin number.
• - value: HIGH (which is 5V on Uno) or LOW (which is 0V).
• - Example: digitalWrite(7, HIGH); // Makes pin 7 output 5V.

• digitalRead(pin)
• - Purpose: Reads the state of a pin configured as INPUT.
• - pin: the pin number to read from.
• - Returns: HIGH if the pin detects a high voltage, or LOW if it detects a low voltage.
• - Example: int buttonState = digitalRead(2); // Reads the voltage on pin 2.

• analogRead(pin)
• - Purpose: Reads an analog voltage from an analog input pin (A0-A5 on Uno).
• - It converts the voltage (from 0V to 5V) into a number.
• - The number ranges from 0 (for 0V) to 1023 (for 5V).
• - pin: the analog pin to read (e.g., A0).
• - Example: int sensorValue = analogRead(A0); // Reads the value from a sensor on A0.

• analogWrite(pin, value)
• - Purpose: Outputs a pseudo-analog signal using PWM (Pulse Width Modulation).
• - Used for tasks like controlling LED brightness or motor speed.
• - pin: must be a PWM-capable pin (marked with ~ on Uno, like 3, 5, 6, 9, 10, 11).
• - value: a number from 0 (0% duty cycle, always off) to 255 (100% duty cycle, always on).
• - Example: analogWrite(9, 128); // Sets the output on pin 9 to 50% duty cycle.

• analogReference(type)
• - Purpose: Configures the maximum voltage reference for analogRead().
• - type: Can be DEFAULT (the default 5V), INTERNAL (a built-in 1.1V reference on Uno), or EXTERNAL.
• - Using INTERNAL is good for getting more precise readings from low-voltage sensors.
• - Example: analogReference(INTERNAL);

• 2. Character Functions
• Functions for testing and classifying single characters.
• These are helpful when processing text or data from serial communication.
• They return true (non-zero) or false (zero).

• isAlpha(char): Checks if a character is an alphabet (a-z, A-Z).
• isAlphaNumeric(char): Checks if a character is an alphabet or a digit.
• isDigit(char): Checks if a character is a numeric digit (0-9).
• isHexadecimalDigit(char): Checks for a valid hex digit (0-9, a-f, A-F).
• isSpace(char): Checks if a character is a space (' ').
• isWhitespace(char): Checks for whitespace (space, tab, newline).

• isUpperCase(char): Checks for an uppercase letter (A-Z).
• isLowerCase(char): Checks for a lowercase letter (a-z).
• - Example: if (isDigit(myChar)) { ... }

• 3. Math Functions
• A collection of standard mathematical functions.

• abs(x)
• - Purpose: Returns the absolute (non-negative) value of number x.
• - Example: abs(-50) returns 50.

• constrain(x, a, b)
• - Purpose: Keeps a number x within a defined range from a to b.
• - x: the number to constrain.
• - a: the lower bound of the range.
• - b: the upper bound of the range.
• - If x is below a, it returns a. If x is above b, it returns b.
• - Example: int value = constrain(analogRead(A0), 0, 255);

• max(x, y)
• - Purpose: Returns the larger of the two numbers, x and y.
• - Example: int largerValue = max(10, 25); // largerValue is 25.

• min(x, y)
• - Purpose: Returns the smaller of the two numbers, x and y.
• - Example: int smallerValue = min(10, 25); // smallerValue is 10.

• pow(base, exponent)
• - Purpose: Calculates the value of base raised to the power of exponent.
• - Returns a floating-point number (double).
• - Example: double result = pow(2, 3); // result is 8.0.

• sqrt(x)
• - Purpose: Calculates the square root of a number x.
• - Returns a floating-point number (double).
• - Example: double result = sqrt(16); // result is 4.0.

7.) 3.7 LED Blinking using Arduino

3.7 LED Blinking using Arduino

1- Introduction to LED Blinking
• The **Hello, World!** program for hardware and microcontrollers.
• A fundamental first step in learning embedded programming.
• It confirms your Arduino board is working correctly.
• It verifies that your programming environment (IDE) is set up properly.
• Teaches the basic concept of controlling an output device.
• You learn to send a signal from the microcontroller to the physical world.

2- Core Concept: Digital Output
• Digital signals have only two states; HIGH or LOW.
• In Arduino Uno; HIGH is represented by 5 Volts (5V).
• In Arduino Uno; LOW is represented by 0 Volts (0V).
• An LED (Light Emitting Diode) is a digital output device.
• It is either ON (when it receives a HIGH signal) or OFF (when it receives a LOW signal).
• We use Arduino's digital pins to send these HIGH and LOW signals.

3- Required Hardware Components

- 1 x Arduino Uno R3 board.
- 1 x LED (any color; e.g., red).
- 1 x Resistor (220 Ohm or 330 Ohm is common).
- 1 x Breadboard (solderless).
- 2 x Jumper wires (male-to-male).

4- Understanding the Components

- LED (Light Emitting Diode)
- A special type of diode that glows when current passes through it.
- It is a polarized component; has a positive and a negative terminal.
- Anode: The positive terminal; it is the longer leg.
- Cathode: The negative terminal; it is the shorter leg.
- The flat edge on the LED's plastic casing also indicates the cathode side.
- Current must flow from Anode (+) to Cathode (-) for it to light up.

- Resistor
- An electronic component that limits the flow of electric current.
- Why it is essential: To protect the LED from excessive current.
- Arduino's digital pins can supply about 40mA of current.
- Most small LEDs require only about 20mA.
- Without a resistor; the LED can burn out instantly.
- The resistor also protects the Arduino's digital pin from damage.
- The value (e.g., 220 Ohm) is chosen based on Ohm's Law (V=IR) to allow a safe current.

- Breadboard
- A board for making temporary circuits without needing to solder.
- It has internal connections.
- Vertical columns (terminal strips) are connected together.
- Horizontal rows at the top and bottom (power rails) are connected together.
- Used for quick prototyping and testing of circuits.

5- The Circuit Connection

- A simple series circuit is created.
- Goal: Connect a digital pin to the LED's anode; and the LED's cathode to ground (GND).
- The resistor can be placed either before or after the LED in the series.

- Step-by-Step Connection:
- 1. Ensure the Arduino is not powered.
- 2. Insert the LED onto the breadboard. Note which leg is longer (anode).
- 3. Connect a jumper wire from the GND pin on the Arduino to the blue negative rail on the breadboard.
- 4. Connect the LED's shorter leg (cathode) to the same negative rail on the breadboard.
- 5. Connect one leg of the 220 Ohm resistor to the same breadboard column as the LED's longer leg (anode).
- 6. Connect the other leg of the resistor to a different, empty column on the breadboard.
- 7. Take another jumper wire. Connect it from Arduino's digital pin 13 to the breadboard column where the resistor ends.

- Path of Current:
- Arduino Pin 13 -> Jumper Wire -> Resistor -> LED Anode (+) -> LED Cathode (-) -> Jumper Wire -> Arduino GND pin.
- This completes the circuit.

6- The Arduino Sketch (The Code)

- An Arduino program is called a **sketch**.
- It has two main parts: setup() and loop().

• A. Global Variable Declaration
• It's good practice to declare the pin number as a variable.
• This makes the code readable and easy to modify.
• Example:
• int ledPin = 13;
• This line creates an integer variable named ledPin and assigns it the value 13.
• Now you can use ledPin instead of 13 throughout your code.

• B. The setup() Function
• This function runs only once when the Arduino is powered on or reset.
• It is used for initialization and configuration.
• We need to tell the Arduino that ledPin will be used as an output pin.
• We use the pinMode() function for this.
• Syntax: pinMode(pinNumber, mode);
• pinNumber is the pin we want to configure (here, ledPin).
• mode can be INPUT or OUTPUT.
• Our code in setup:
• void setup() {
• pinMode(ledPin, OUTPUT);
• }
• This line configures digital pin 13 as an output pin.

• C. The loop() Function
• This function runs continuously after the setup() function is finished.
• It repeats its code over and over again.
• This is where the main logic of the program goes.
• The logic for blinking is: Turn ON -> Wait -> Turn OFF -> Wait.

• Step 1: Turn the LED ON.
• We use the digitalWrite() function to send a HIGH (5V) signal.
• Syntax: digitalWrite(pinNumber, value);
• value can be HIGH or LOW.
• Code: digitalWrite(ledPin, HIGH);

• Step 2: Wait for some time.
• We use the delay() function to pause the program.
• Syntax: delay(milliseconds);
• The argument is in milliseconds (1000 ms = 1 second).
• Code: delay(1000); // Wait for one second

• Step 3: Turn the LED OFF.
• We use digitalWrite() again, but this time send a LOW (0V) signal.
• Code: digitalWrite(ledPin, LOW);

• Step 4: Wait again.
• This creates the duration for which the LED stays off.
• Code: delay(1000); // Wait for one second

• Complete loop() function:
• void loop() {
• digitalWrite(ledPin, HIGH); // Turn the LED on
• delay(1000); // Wait for a second
• digitalWrite(ledPin, LOW); // Turn the LED off
• delay(1000); // Wait for a second
• }
• The loop repeats, creating the blinking effect.

7- Full Arduino Sketch for Blinking an LED
`

```
// Global variable for the LED pin
int ledPin = 13;

// The setup function runs once when you press reset or power the board
void setup() {
// Initialize the digital pin as an output.
pinMode(ledPin, OUTPUT);
}

// The loop function runs over and over again forever
void loop() {
digitalWrite(ledPin, HIGH); // turn the LED on (HIGH is the voltage level)
delay(1000); // wait for a second
digitalWrite(ledPin, LOW); // turn the LED off by making the voltage LOW
delay(1000); // wait for a second
}
```

8- Uploading and Testing

- 1. Connect your Arduino Uno to your computer using a USB cable.
- 2. Open the Arduino IDE software.
- 3. Copy and paste the full sketch into the IDE.
- 4. Go to Tools > Board and select **Arduino Uno**.
- 5. Go to Tools > Port and select the correct COM port for your Arduino.
- 6. Click the **Upload** button (an arrow pointing right).
- 7. The IDE will compile the sketch and upload it to the Arduino.
- 8. Once done, the external LED connected to pin 13 on your breadboard should start blinking.

9- Using the On-board LED

- The Arduino Uno board has a built-in LED.
- This LED is internally connected to digital pin 13.
- It is usually labeled 'L' on the board.
- You can test the blinking sketch without any external circuit.
- Just upload the code with `ledPin = 13;`.
- The small 'L' LED on the Arduino board itself will start to blink.
- This is very useful for quick code verification.

10- Practical Exercises and Modifications

- Faster Blinking: Change both `delay(1000);` to `delay(250);`.
- Asymmetric Blinking: Make it stay ON for 2 seconds and OFF for 500 milliseconds.
- `digitalWrite(ledPin, HIGH);`
- `delay(2000);`
- `digitalWrite(ledPin, LOW);`
- `delay(500);`
- Change the Pin: Change `int ledPin = 13;` to `int ledPin = 8;` and move the jumper wire from pin 13 to pin 8 on the Arduino.
- These simple changes help solidify your understanding of the code's control over the hardware.

8.) 3.8 Serial Communication Functions: Serial, available, begin, end, print, println, write, read, readBytes, readString.

3.8 Serial Communication Functions

A. Introduction to Serial Communication

- Serial communication is a process; of sending data one bit at a time; over a single wire or channel.

• In Arduino, it's the primary way to talk to a computer; for debugging or sending sensor data.
• The Arduino Uno board uses digital pins 0 (RX - Receive) and 1 (TX - Transmit) for this.
• RX receives data; TX transmits data.
• The USB cable connects these pins to your computer; through an onboard USB-to-serial converter chip.
• This allows you to use the **Serial Monitor** in the Arduino IDE.

B. The 'Serial' Object

• In Arduino programming; Serial is a built-in object.
• An object is a programming tool; that bundles variables and functions together.
• You don't need to create it; it's always available.
• We use it to call all serial communication functions; for example, Serial.begin() or Serial.print().

C. Setup and Control Functions

1. Serial.begin(speed)
• Purpose: To start or initialize serial communication.
• It's the first serial function you must call.
• Typically placed inside the void setup() function.
• Parameter speed: This is the baud rate; or data transfer speed in bits per second (bps).
• Common baud rate: 9600.
• Important: The baud rate in your code must match the baud rate set in the Serial Monitor.
• Analogy: It's like picking up a phone and agreeing on a language and speed to talk at.
• Example: void setup() { Serial.begin(9600); }

2. Serial.end()
• Purpose: To stop or disable serial communication.
• It's the opposite of Serial.begin().
• After calling Serial.end(), pins 0 and 1 can be used as regular digital I/O pins.
• Use case: Useful in low-power applications or when you need every pin for other tasks.
• Example: void loop() { /* some condition */ Serial.end(); }

D. Sending Data (From Arduino to Computer)

1. Serial.print(data)
• Purpose: Sends data to the serial port in human-readable ASCII text.
• It can send different data types: numbers, characters, strings.
• Example 1: Serial.print(78); will send the characters '7' and '8'.
• Example 2: Serial.print(1.23456); will send '1', '.', '2', '3'. (by default, 2 decimal places).
• Example 3: Serial.print(**Hello**); will send the characters 'H', 'e', 'l', 'l', 'o'.
• The cursor in the Serial Monitor stays on the same line after printing.

2. Serial.println(data)
• Purpose: Same as Serial.print(), but with one key difference.
• After sending the data; it also sends a **carriage return** and **newline** character.
• This moves the cursor to the beginning of the next line in the Serial Monitor.
• 'ln' stands for 'line'.
• Very useful for formatting output to be easily readable.
• Example:

```
Serial.print(Value: );
Serial.println(analogRead(A0));
```

• Output in Serial Monitor: Value: 452 (and cursor moves to the next line).

3. Serial.write(data)
• Purpose: Sends raw binary data, one byte at a time.
• It does not convert numbers into text characters.
• print() is for humans; write() is for machines.
• Example Comparison:

• Serial.print(65); sends the ASCII text **65** (two bytes: character '6', character '5').
• Serial.write(65); sends the single binary byte value 65; which corresponds to the ASCII character 'A'.
• Use write() when you need to send specific byte values to another device; not for display in the Serial Monitor.

E. Checking and Reading Data (From Computer to Arduino)

1. Serial.available()
• Purpose: To check if any data has arrived from the computer and is waiting to be read.
• It checks a temporary storage area called the serial buffer (size: 64 bytes).
• Return Value: It returns the number of bytes (characters) available in the buffer.
• If the buffer is empty, it returns 0.
• How to use: Always use this inside an if condition before trying to read data.
• Example: if (Serial.available() > 0) { // data is ready, now read it }

2. Serial.read()
• Purpose: Reads one single byte (character) of incoming data from the serial buffer.
• Once a byte is read, it is removed from the buffer.
• Return Value: Returns the first available byte of data as an int.
• If no data is available, it returns -1.
• This is the most fundamental way to read incoming data.
• Example:
`
if (Serial.available() > 0) {
char incomingByte = Serial.read();
Serial.print(**I received:**);
Serial.println(incomingByte);
}
`

3. Serial.readBytes(buffer, length)
• Purpose: Reads a fixed number of bytes from the serial buffer into an array.
• Parameter buffer: An array of char or byte to store the data.
• Parameter length: The number of bytes you want to read.
• Behavior: This function will wait (block the code) until length bytes have been received OR a timeout occurs (default 1 second).
• Return Value: The number of bytes that were actually read and placed in the buffer.
• Useful for protocols where you know the exact message length.
• Example:
```
char myBuffer[10];
if (Serial.available() >= 10) {
Serial.readBytes(myBuffer, 10);
}
```

4. Serial.readString()
• Purpose: Reads characters from the serial buffer into a String object.
• Behavior: It also waits (blocks) for a timeout to occur before it returns the string.
• It reads until it times out; making it easy to get a whole line of text.
• This is easier to use than read() for text commands.
• Caution: The String object can use more memory and be slower than reading byte-by-byte with read(). It is best avoided in large, complex programs.
• Example:
`
if (Serial.available() > 0) {
String command = Serial.readString();
if (command == **ON**) {
digitalWrite(13, HIGH);
}

}
`

F. Summary and Key Points for Revision

- Setup: Always start with Serial.begin(9600); in setup().
- Sending Data:
- print(): Human-readable text, stays on the same line.
- println(): Human-readable text, moves to the next line.
- write(): Raw binary byte, for machine communication.
- Receiving Data:
- Always check with if (Serial.available() > 0) first.
- read(): Reads one character at a time. The most reliable method.
- readBytes(): Reads a fixed number of bytes into an array.
- readString(): Reads a whole line of text into a String object. Easiest but can be inefficient.
- Serial Monitor: Ensure the baud rate in the bottom-right corner matches the one in Serial.begin().
- Pins 0 & 1: Do not use these for digital I/O (like with LEDs or buttons) when Serial communication is active. They are reserved for RX and TX.